CSCE 775: Deep Reinforcement Learning
Homework #1

Instructor: Dr. Forest Agostinelli
University of South Carolina
Solutions by: JC Vaught

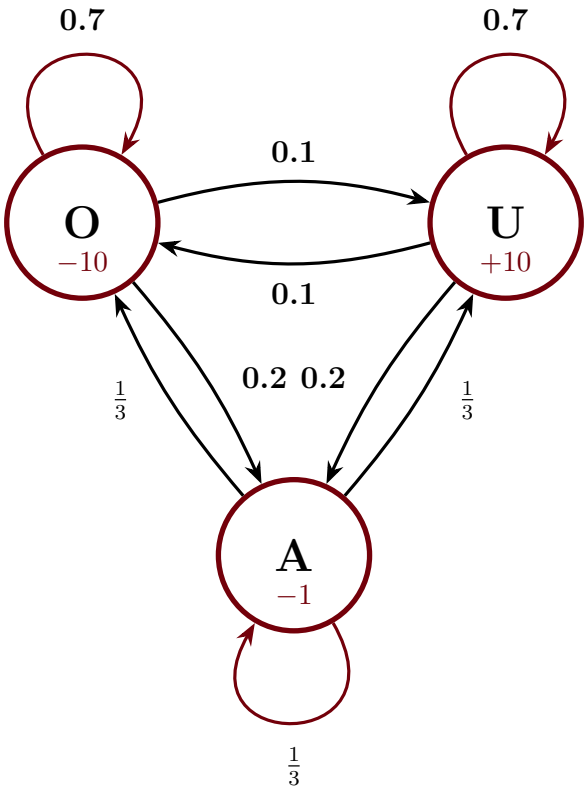
Spring 2026

Table of Contents

Problem 1: Markov Reward Processes (20 pts)

Task: Implement the `mrp` function. Given the definition of a Markov reward process, compute the state-values (expected future reward) for all states.

MRP Diagram



Problem Formulation

The state space is $\mathcal{S} = \{O, U, A\}$ with rewards $R(O) = -10$, $R(U) = +10$, $R(A) = -1$. The transition probability matrix is:

$$\mathcal{P} = \begin{pmatrix} 0.7 & 0.1 & 0.2 \\ 0.1 & 0.7 & 0.2 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{pmatrix}, \qquad \mathbf{R} = \begin{pmatrix} -10 \\ +10 \\ -1 \end{pmatrix}$$

Solution Approach: Matrix Inversion

Step 1: Bellman Equation in Matrix Form

The value function satisfies:

$$\mathbf{V} = \mathbf{R} + \gamma \mathcal{P}\mathbf{V} \quad \Longrightarrow \quad \mathbf{V} = (\mathbf{I} - \gamma \mathcal{P})^{-1}\mathbf{R}$$

Step 2: Compute I – gamma P for gamma

$$\mathbf{I} - 0.9\mathcal{P} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - 0.9 \begin{pmatrix} 0.7 & 0.1 & 0.2 \\ 0.1 & 0.7 & 0.2 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{pmatrix} = \begin{pmatrix} 0.37 & -0.09 & -0.18 \\ -0.09 & 0.37 & -0.18 \\ -0.3 & -0.3 & 0.7 \end{pmatrix}$$

Step 3: Solve the Linear System

Computing $\mathbf{V} = (\mathbf{I} - \gamma\mathcal{P})^{-1}\mathbf{R}$ via matrix inversion or a linear solver yields:

Results

For $\gamma = 0.9$:

$$V(O) \approx -23.75, \qquad V(U) \approx +32.01, \qquad V(A) \approx +1.04$$

Interpretation:

- $V(O) < 0$: The system tends to stay in O (70% self-loop), accumulating -10 rewards.
- $V(U) > 0$: The system tends to stay in U (70% self-loop), accumulating $+10$ rewards.
- $V(A) \approx 0$: Despite the -1 reward, there is a $\frac{1}{3}$ chance of reaching the valuable state U.

Verification Across Discount Factors

γ	$\mathbf{V}(\mathbf{O})$	$\mathbf{V}(\mathbf{U})$	$\mathbf{V}(\mathbf{A})$
0	−10.00	+10.00	−1.00
0.1	−10.63	+10.63	−1.00
0.9	−23.75	+32.01	+1.04
0.999	−163.30	+343.37	+88.03

Implementation

The `mrp` function uses NumPy's `np.linalg.solve` to solve $(\mathbf{I} - \gamma\mathcal{P})\mathbf{V} = \mathbf{R}$:

```
import numpy as np

def mrp(P, R, gamma):
    I = np.eye(P.shape[0])
    return np.linalg.solve(I - gamma * P, R)
```

Problem 2.1: Dynamic Programming (40 pts)

Task: Implement the `dynamic_programming` function. Use any tabular dynamic programming algorithm to exactly compute the optimal state-value function and an optimal policy for the AI Farm environment.

Algorithm: Value Iteration

Value iteration repeatedly applies the Bellman optimality update until convergence:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V_k(s') \right]$$

The optimal policy is then:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V(s') \right]$$

Step 1: Initialize

Set $V(s) = 0$ for all states s. Choose convergence threshold $\theta = 10^{-8}$.

Step 2: Value Iteration Loop

For each state s and each action a:

1. Call `env.state_action_dynamics(s, a)` to get $r(s,a)$, next states $\{s'\}$, and probabilities $\{P(s'|s,a)\}$.
2. Compute $Q(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V(s')$.
3. Update $V(s) = \max_a Q(s,a)$.

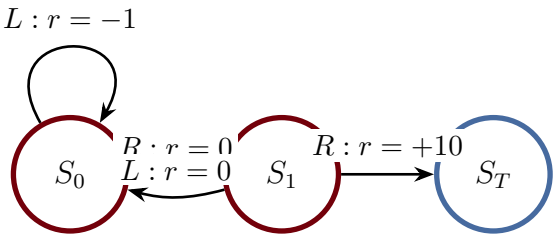
Repeat until $\max_s |V_{\text{new}}(s) - V_{\text{old}}(s)| < \theta$.

Step 3: Extract Policy

For each state s, compute the action probabilities. For the greedy deterministic policy, assign probability 1 to $\arg\max_a Q(s,a)$ and 0 to all other actions.

Worked Example: 2-State MDP

Consider a simple MDP with states S_0, S_1 and terminal state S_T, $\gamma = 1$:



Value Iteration:

Iter	$\mathbf{V}(\mathbf{S_0})$	$\mathbf{V}(\mathbf{S_1})$	$\boldsymbol{\pi}(\boldsymbol{S_0})$	$\boldsymbol{\pi}(\boldsymbol{S_1})$
0	0	0	—	—
1	0	10	R	R
2	10	10	R	R
3	10	10	R	R

Results

Converged: $V^*(S_0) = 10$, $V^*(S_1) = 10$. Optimal policy: $S_0 \rightarrow$ right $\rightarrow S_1 \rightarrow$ right $\rightarrow S_T$ collecting $+10$ reward.

Implementation Sketch

```
def dynamic_programming(env, states, state_values, gamma):
    theta = 1e-8
    while True:
        delta = 0
        for s in states:
            actions = env.get_actions(s)
            if not actions:
                continue
            v_old = state_values[s]
            q_values = []
            for a in actions:
                r, next_states, probs = env.state_action_dynamics(s, a)
                q = r + gamma * sum(p * state_values[ns]
                                    for ns, p in zip(next_states, probs))
                q_values.append(q)
            state_values[s] = max(q_values)
            delta = max(delta, abs(v_old - state_values[s]))
        if delta < theta:
            break

    # Extract policy
    policy = {}
    for s in states:
        actions = env.get_actions(s)
        action_probs = [0.0] * len(actions)
        if actions:
            best_q = float('-inf')
            best_idx = 0
            for i, a in enumerate(actions):
                r, next_states, probs = env.state_action_dynamics(s, a)
                q = r + gamma * sum(p * state_values[ns]
                                    for ns, p in zip(next_states, probs))
                if q > best_q:
                    best_q = q
                    best_idx = i
            action_probs[best_idx] = 1.0
        policy[s] = action_probs

    return state_values, policy
```

Problem 2.2: Model-Free Reinforcement Learning (40 pts)

Task: Implement the `model_free` function. Use a tabular model-free RL algorithm to approximate the optimal action-value function. **Cannot** use `env.state_action_dynamics`.

Algorithm: Q-Learning

Q-learning is an off-policy TD control algorithm. The update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

with ε-greedy exploration:

$$a = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$$

Step 1: Initialize

Set $Q(s,a) = 0$ for all state-action pairs. Choose hyperparameters:

- Learning rate $\alpha \in (0,1]$ (e.g., $\alpha = 0.1$)
- Exploration rate $\varepsilon \in (0,1]$ (e.g., $\varepsilon = 0.3$, decaying)
- Number of episodes (e.g., $N = 10000$)

Step 2: Episode Loop

For each episode:

1. Sample start state $s \leftarrow$ `env.sample_start_state()`.
2. While s is not terminal:
   (a) Select action a via ε-greedy w.r.t. $Q(s,\cdot)$.
   (b) Take action: $(s', r) \leftarrow$ `env.sample_transition(s, a)`.
   (c) Update: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.
   (d) $s \leftarrow s'$.

Worked Example: 2-State MDP with Q-Learning

Same 2-state MDP as before. $\alpha = 0.1$, $\gamma = 1.0$, $\varepsilon = 0.3$.
Initialize: $Q(S_0, L) = Q(S_0, R) = Q(S_1, L) = Q(S_1, R) = 0$.

Episode 1

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0,R) = 0 + 0.1[0 + 0 - 0] = 0$
2	S_1	R	S_T	10	$Q(S_1,R) = 0 + 0.1[10 + 0 - 0] = 1.0$

Episode 2

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0, R) = 0 + 0.1[0 + 1.0 - 0] = 0.1$
2	S_1	R	S_T	10	$Q(S_1, R) = 1.0 + 0.1[10 + 0 - 1.0] = 1.9$

Episode 3

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0, R) = 0.1 + 0.1[0 + 1.9 - 0.1] = 0.28$
2	S_1	R	S_T	10	$Q(S_1, R) = 1.9 + 0.1[10 + 0 - 1.9] = 2.71$

Results

After many episodes: $Q(S_0, R) \to 10$, $Q(S_1, R) \to 10$, matching the DP solution. The greedy policy $\pi(s) = \arg\max_a Q(s, a)$ yields $S_0 \to R$, $S_1 \to R$.

Implementation Sketch

```
import random

def model_free(env, action_values, gamma):
    alpha = 0.1
    epsilon = 1.0
    epsilon_decay = 0.9995
    epsilon_min = 0.01
    num_episodes = 50000

    for ep in range(num_episodes):
        state = env.sample_start_state()
        while True:
            actions = env.get_actions(state)
            if not actions:
                break
            # Epsilon-greedy
            if random.random() < epsilon:
                idx = random.randrange(len(actions))
            else:
                idx = max(range(len(actions)),
                          key=lambda i: action_values[state][i])
            action = actions[idx]
            next_state, reward = env.sample_transition(state, action)

            # Q-learning update
            next_actions = env.get_actions(next_state)
            max_q_next = max(action_values[next_state]) if next_actions else 0.0
            td_target = reward + gamma * max_q_next
            td_error = td_target - action_values[state][idx]
            action_values[state][idx] += alpha * td_error

            state = next_state

        epsilon = max(epsilon_min, epsilon * epsilon_decay)

    return action_values
```